



**Brit College of Engineering & Technology
(BCET)**

Data Analytics Logbook

Research Project-5 Data Science and AI

Submitted by: Md.Akib Sultan

Student ID:

Email: akibsultan1045@gmail.com

Google colab: <https://colab.research.google.com/drive/1MLexcph4H8ehCb9C1Mz2uz-Wky9PDaTy?usp=sharing>

Unsupervised learning is a powerful branch of machine learning that deals with finding hidden patterns and structures in unlabelled data. Unlike supervised learning, where models are trained on data with known outcomes (labels), unsupervised learning algorithms work autonomously to discover inherent groupings, dimensions, or anomalies within the data itself.

1. Introduction to Unsupervised Learning

Imagine you have a vast collection of customer data, but no information about how they might be grouped. Unsupervised learning helps you discover natural segments among these customers based on their purchasing habits, demographics, or other attributes. Key applications include:

- **Clustering:** Grouping similar data points together (e.g., customer segmentation, document categorization).
- **Dimensionality Reduction:** Reducing the number of features while preserving important information (e.g., for visualization, noise reduction, or improving model performance).
- **Anomaly Detection:** Identifying unusual data points that deviate significantly from the norm (e.g., fraud detection, network intrusion detection).

In this tutorial, we will primarily focus on **clustering** and use **dimensionality reduction** as a preparatory step for better visualization.

2. The Dataset: Wholesale Customers

For this tutorial, we'll use the **Wholesale Customers Data Set**. This dataset contains the annual spending (in monetary units) of various clients on different product categories. Our goal will be to segment these customers into groups based on their spending behavior.

Attributes Information:

1. Channel: Customer's Channel (Retail or Horeca)
2. Region: Customer's Region (Lisbon, Oporto, or Other)
3. Fresh: Annual spending (m.u.) on fresh products (e.g. fruits, vegetables)
4. Milk: Annual spending (m.u.) on milk products (e.g. powder milk, butter,cheese)

5. Grocery: Annual spending (m.u.) on grocery products (e.g. frozen foods, prepared meals)
6. Frozen: Annual spending (m.u.) on frozen products (e.g. frozen fruits, vegetables)
7. Detergents_Paper: Annual spending (m.u.) on detergents and paper products
8. Delicassen: Annual spending (m.u.) on delicatessen products (e.g. prepared meats, seafood)

Step 1: Import Libraries

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
```

```
from sklearn.metrics import silhouette_score
```

```
from sklearn.decomposition import PCA
```

```
from scipy.cluster.hierarchy import dendrogram, linkage
```

```
# Ignore warnings for cleaner output
```

```
import warnings
```

```
warnings.filterwarnings('ignore')
```

```
# Set aesthetic style for plots
```

```
sns.set_style("whitegrid")
```

Step 2: Load the Dataset

```
# The dataset is often available as a CSV.
```

```
# You might need to upload it to Colab's session storage
```

```
# or directly load from a URL if available.
```

```
# For simplicity, we'll assume it's in the same directory or provide a direct link if possible.
```

As of 2023, the UCI link provides a .csv directly downloadable

If having issues with direct download, download manually and upload to Colab

```
url = "https://archive.ics.uci.edu/ml/machine-learningdatabases/00292/Wholesale%20customers%20data.csv"
```

try:

```
df = pd.read_csv(url)
```

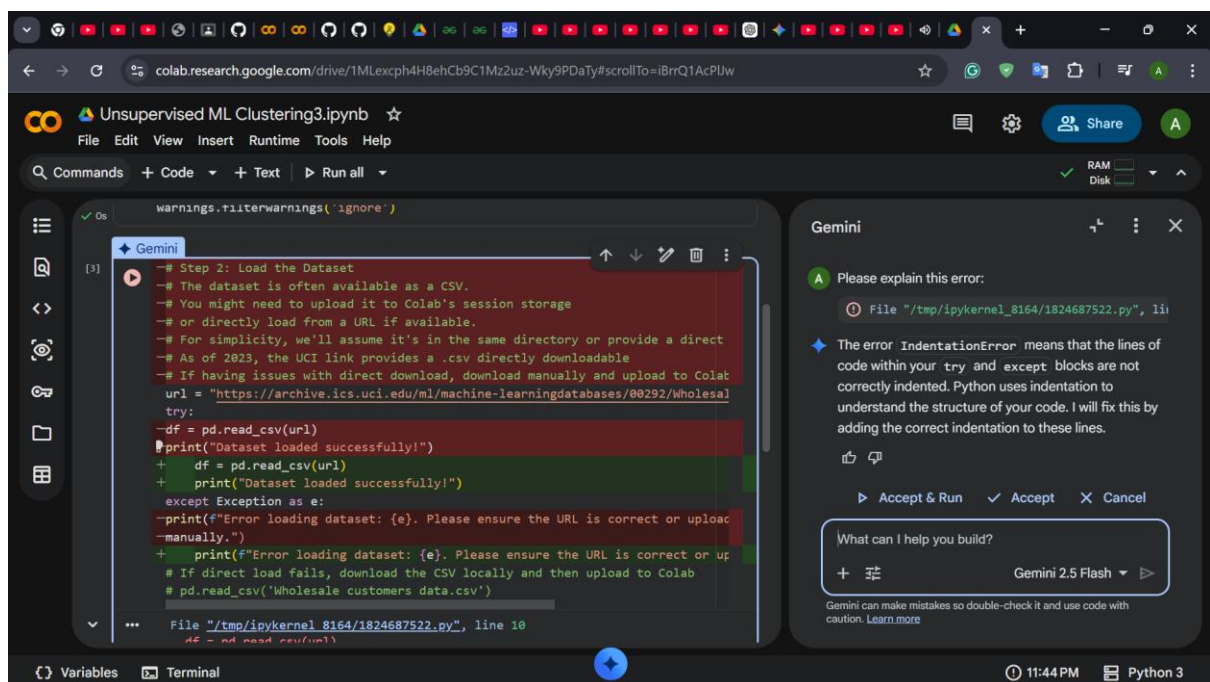
```
print("Dataset loaded successfully!")
```

except Exception as e:

```
print(f"Error loading dataset: {e}. Please ensure the URL is correct or upload the file manually.")
```

If direct load fails, download the CSV locally and then upload to Colab

```
# pd.read_csv('Wholesale customers data.csv')
```



At this point the existing code shows error while do run in directory. So I get help from Ai to correct my code. Still the datasets couldn't find so I have to insert manually. After Correcting:

Example: Load a CSV file from Google Drive

Replace 'path/to/your/file.csv' with the actual path to your data on Google Drive

```
drive_file_path = "/content/Wholesale customers data.csv"
```

try:

```
df = pd.read_csv(drive_file_path)
```

```

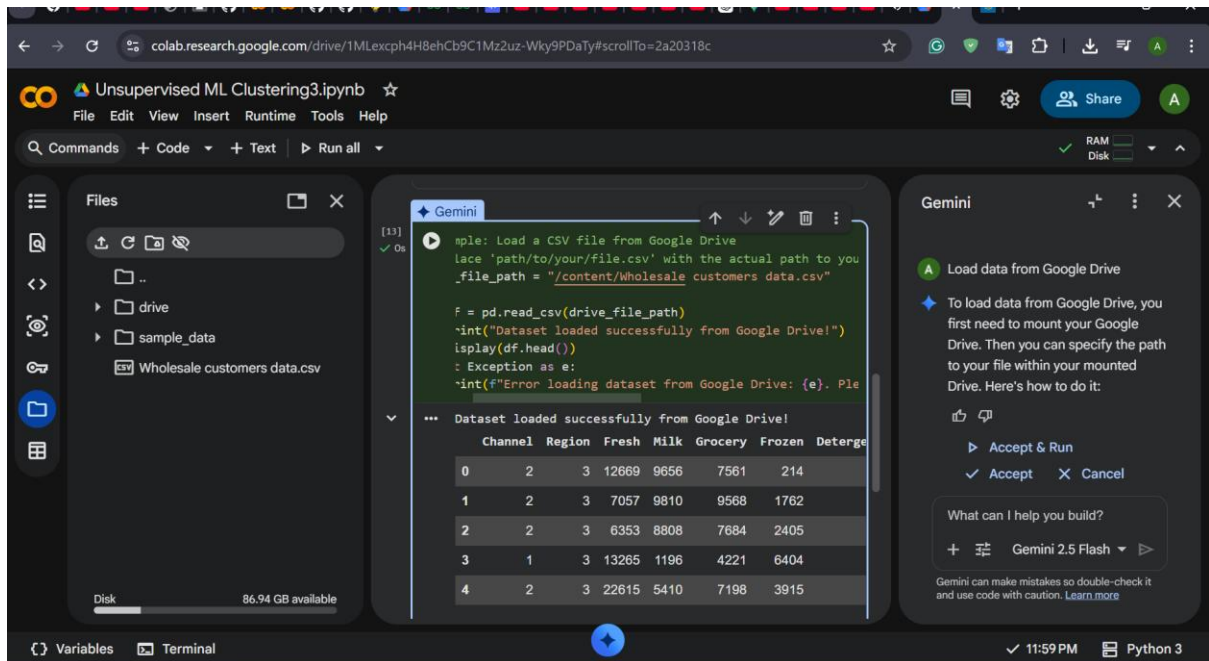
print("Dataset loaded successfully from Google Drive!")

display(df.head())

except Exception as e:

    print(f"Error loading dataset from Google Drive: {e}. Please check the file path and ensure
permissions are correct.")

```



Step 3: Initial Exploration (EDA - Part 1)

```

print("\n--- Dataset Head ---")

print(df.head())

print("\n--- Dataset Info ---")

df.info()

print("\n--- Dataset Description ---")

print(df.describe())

print("\n--- Missing Values ---")

print(df.isnull().sum())

```

```
# Step 3: Initial Exploration (EDA - Part 1)
print("\n--- Dataset Head ---")
print(df.head())
print("\n--- Dataset Info ---")
df.info()
print("\n--- Dataset Description ---")
print(df.describe())
print("\n--- Missing Values ---")
print(df.isnull().sum())

--- Dataset Head ---
   Channel  Region  Fresh  Milk  Grocery  Frozen  Detergents_Paper  Delicassen
0         2       3  12669  9656    7561     214             2674           1338
1         1       3   7057  9810    9568    1762             3293           1776
2         2       3   6353  8808    7684    2405             3516           7844
3         1       3  13265  1196    4221    6404              507           1788
4         2       3  22615  5410    7198    3915             1777           5185

--- Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 440 entries, 0 to 439
Data columns (total 8 columns):
 #   Column        Non-Null Count  Dtype  
---  --
 0   Channel        440 non-null    object  
 1   Region         440 non-null    object  
 2   Fresh          440 non-null    int64   
 3   Milk           440 non-null    int64   
 4   Grocery        440 non-null    int64   
 5   Frozen         440 non-null    int64   
 6   Detergents_Paper 440 non-null    int64   
 7   Delicassen     440 non-null    int64   
```

Observations from Initial Exploration:

- The Channel and Region columns are categorical, while the rest are numerical.
- `df.describe()` shows wide ranges in spending, indicating potential need for scaling.
- No missing values, which is great!

Step 5. Data Cleaning, Preprocessing & Wrangling

Data Wrangling & Preprocessing Steps:

1. **Drop Channel and Region:** For pure spending-based clustering, we might initially exclude Channel and Region as they are categorical identifiers. If we wanted to cluster *within* regions or channels, we'd keep them and handle them differently (e.g., one-hot encode, but that would also increase dimensionality). For a simpler clustering task focused on spending, we'll drop them for now.
2. **Feature Scaling:** Unsupervised algorithms, especially distance-based ones like K-Means and Hierarchical Clustering, are highly sensitive to the scale of the features. Features with larger values will have a disproportionately higher influence on the distance calculation. We'll use `StandardScaler` to transform the data to have a mean of 0 and a standard deviation of 1.

Step 1: Drop Channel and Region for initial clustering based on spending habits

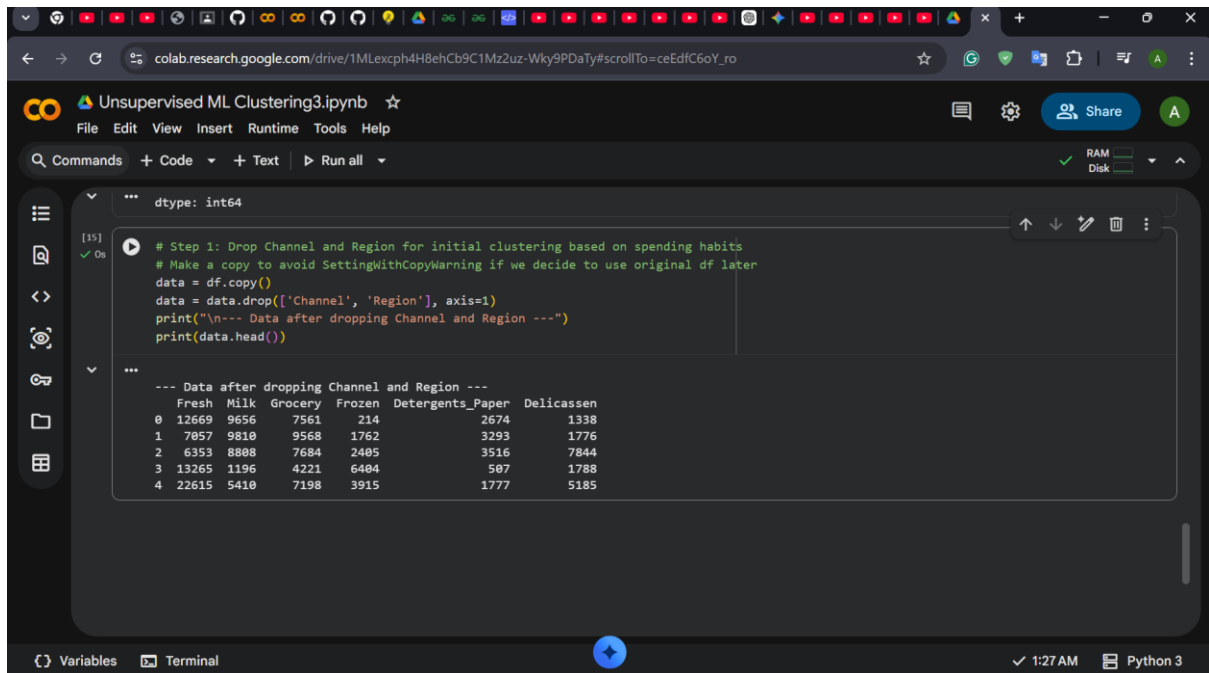
Make a copy to avoid SettingWithCopyWarning if we decide to use original df later

```
data = df.copy()

data = data.drop(['Channel', 'Region'], axis=1)

print("\n--- Data after dropping Channel and Region ---")

print(data.head())
```



The screenshot shows a Google Colab notebook titled "Unsupervised ML Clustering3.ipynb". The code cell contains the following Python code:

```
dtype: int64

# Step 1: Drop Channel and Region for initial clustering based on spending habits
# Make a copy to avoid SettingWithCopyWarning if we decide to use original df later
data = df.copy()
data = data.drop(['Channel', 'Region'], axis=1)
print("\n--- Data after dropping Channel and Region ---")
print(data.head())
```

The output of the code is displayed below the code cell:

```
--- Data after dropping Channel and Region ---
   Fresh  Milk  Grocery  Frozen  Detergents_Paper  Delicassen
0  12669  9656   7561    214          2674         1338
1   7857  9810   9568   1762          3293         1776
2   6353  8808   7684   2405          3516         7844
3  13265  1196   4221   6404           507         1788
4  22615  5410   7198   3915          1777         5185
```

Step 2: Feature Scaling

```
scaler = StandardScaler()

scaled_data = scaler.fit_transform(data)

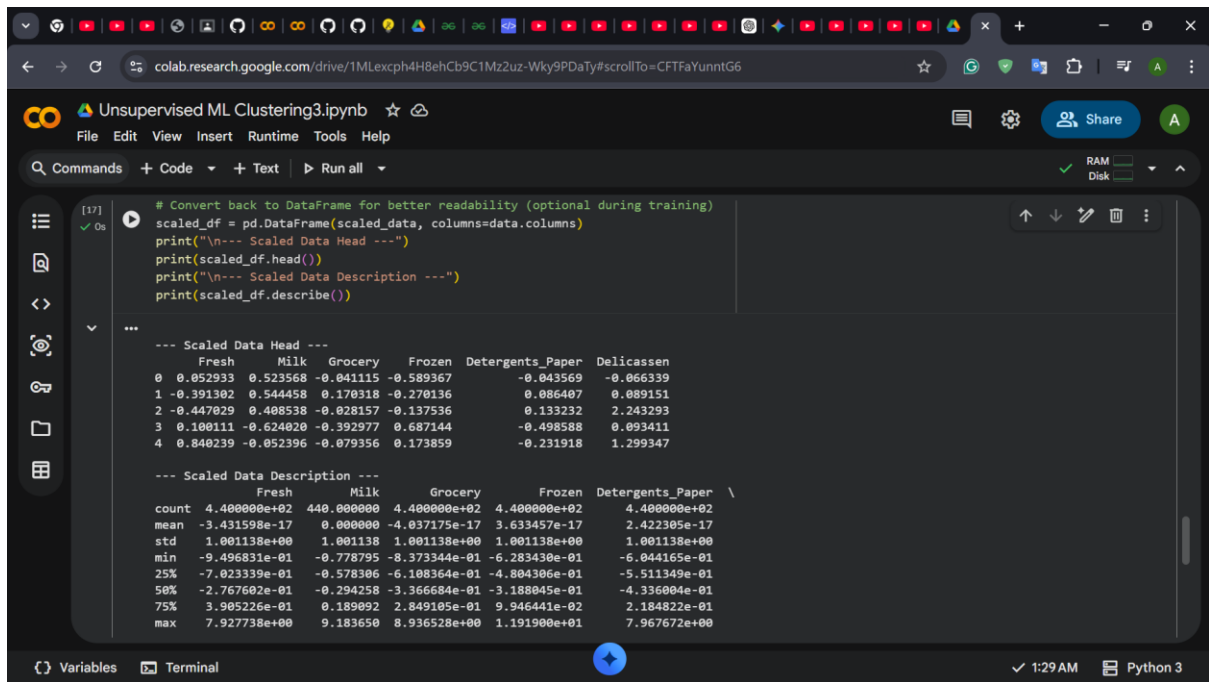
# Convert back to DataFrame for better readability (optional during training)
scaled_df = pd.DataFrame(scaled_data, columns=data.columns)

print("\n--- Scaled Data Head ---")

print(scaled_df.head())

print("\n--- Scaled Data Description ---")

print(scaled_df.describe())
```



```
# Convert back to DataFrame for better readability (optional during training)
scaled_df = pd.DataFrame(scaled_data, columns=data.columns)
print("\n--- Scaled Data Head ---")
print(scaled_df.head())
print("\n--- Scaled Data Description ---")
print(scaled_df.describe())

--- Scaled Data Head ---
   Fresh  Milk  Grocery  Frozen  Detergents_Paper  Delicassen
0  0.052933  0.523568 -0.041115 -0.589367      -0.043569      -0.066339
1 -0.391302  0.544458  0.170318 -0.270136      0.006407      0.089151
2 -0.447029  0.408538 -0.028157 -0.137536      0.133232      2.243293
3  0.100111 -0.624020 -0.392977  0.687144      -0.498588      0.093411
4  0.840239 -0.052396 -0.079356  0.173859      -0.231918      1.299347

--- Scaled Data Description ---
      Fresh      Milk      Grocery      Frozen  Detergents_Paper  \
count  4.400000e+02  440.000000  4.400000e+02  4.400000e+02  4.400000e+02
mean   -3.431598e-17   0.000000  -4.037175e-17  3.633457e-17  2.422305e-17
std     1.001138e+00   1.001138  1.001138e+00  1.001138e+00  1.001138e+00
min    -9.496831e-01  -0.778795  -8.373344e-01  -6.283430e-01  -6.044165e-01
25%    -7.023339e-01  -0.578306  -6.108364e-01  -4.804306e-01  -5.511349e-01
50%    -2.767602e-01  -0.294258  -3.366684e-01  -3.188045e-01  -4.336004e-01
75%     3.905226e-01  0.189092   2.849105e-01  9.946441e-02  2.184822e-01
max     7.927738e+00   9.183650  8.936528e+00  1.191900e+01  7.967672e+00
```

7. Exploratory Data Analysis (EDA) - Deeper Dive

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# Step 1: Distribution of features (after scaling)
```

```
plt.figure(figsize=(15, 10))
```

```
for i, col in enumerate(scaled_df.columns):
```

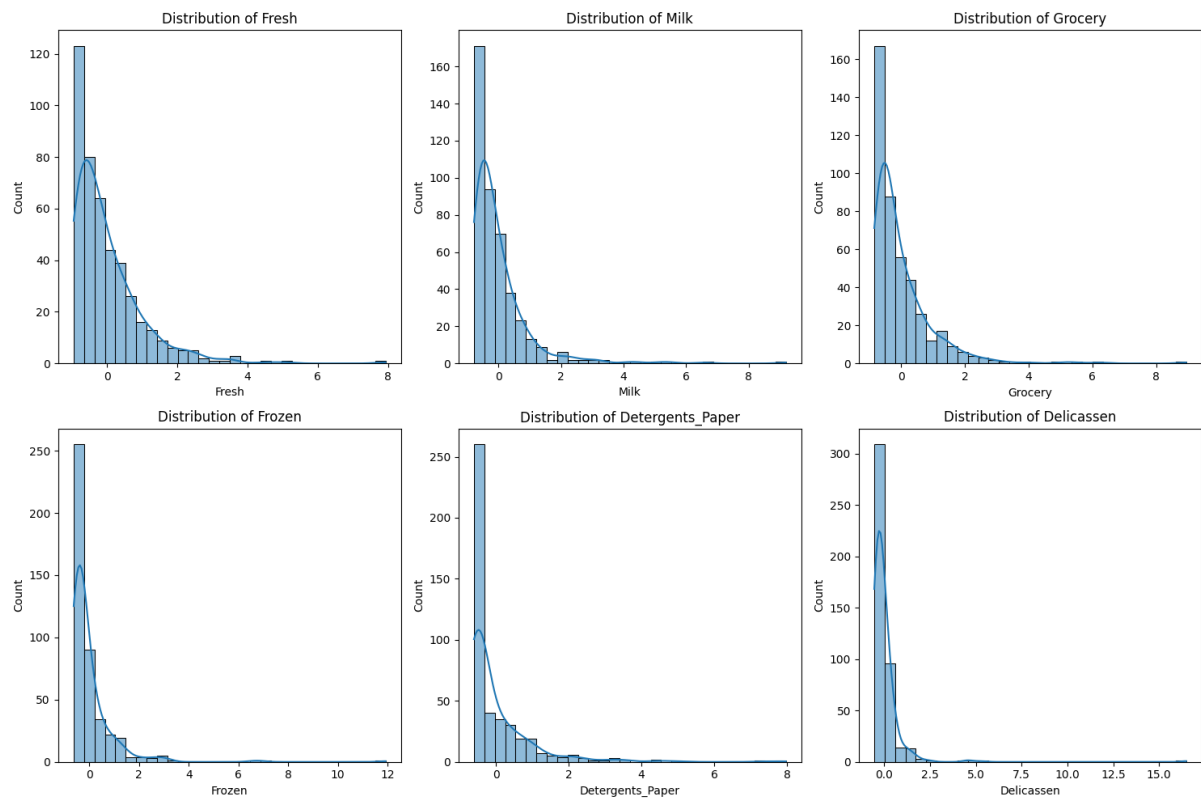
```
    plt.subplot(2, 3, i + 1)
```

```
    sns.histplot(scaled_df[col], kde=True, bins=30)
```

```
    plt.title(f'Distribution of {col}')
```

```
plt.tight_layout()
```

```
plt.show()
```



Step 2: Correlation Matrix (before and after scaling - scaling doesn't change correlation)

```
plt.figure(figsize=(10, 8))
```

```
sns.heatmap(data.corr(), annot=True, cmap='coolwarm', fmt=".2f")
```

```
plt.title('Correlation Matrix of Original Features')
```

```
plt.show()
```



Observations from EDA:

Some features like 'Detergents_Paper' and 'Grocery' are highly correlated, which makes sense.

#The distributions, though standardized, might still show skewness. Clustering algorithms implicitly handle this

by finding natural groupings, but techniques like log transformations could be considered if data is heavily skewed

and specific distribution assumptions are required for other models (not strictly for KMeans/Agglomerative).

8. Unsupervised Learning Algorithms & Training

Step 8.1.1: Determine optimal number of clusters (k) using Elbow Method

```
wcss = [] # Within-Cluster Sum of Squares

for i in range(1, 11):

    kmeans = KMeans(n_clusters=i, init='k-means++', max_iter=300, n_init=10, random_state=42)

    kmeans.fit(scaled_data)

    wcss.append(kmeans.inertia_) # inertia_ is the WCSS

plt.figure(figsize=(10, 6))

plt.plot(range(1, 11), wcss, marker='o', linestyle='--')

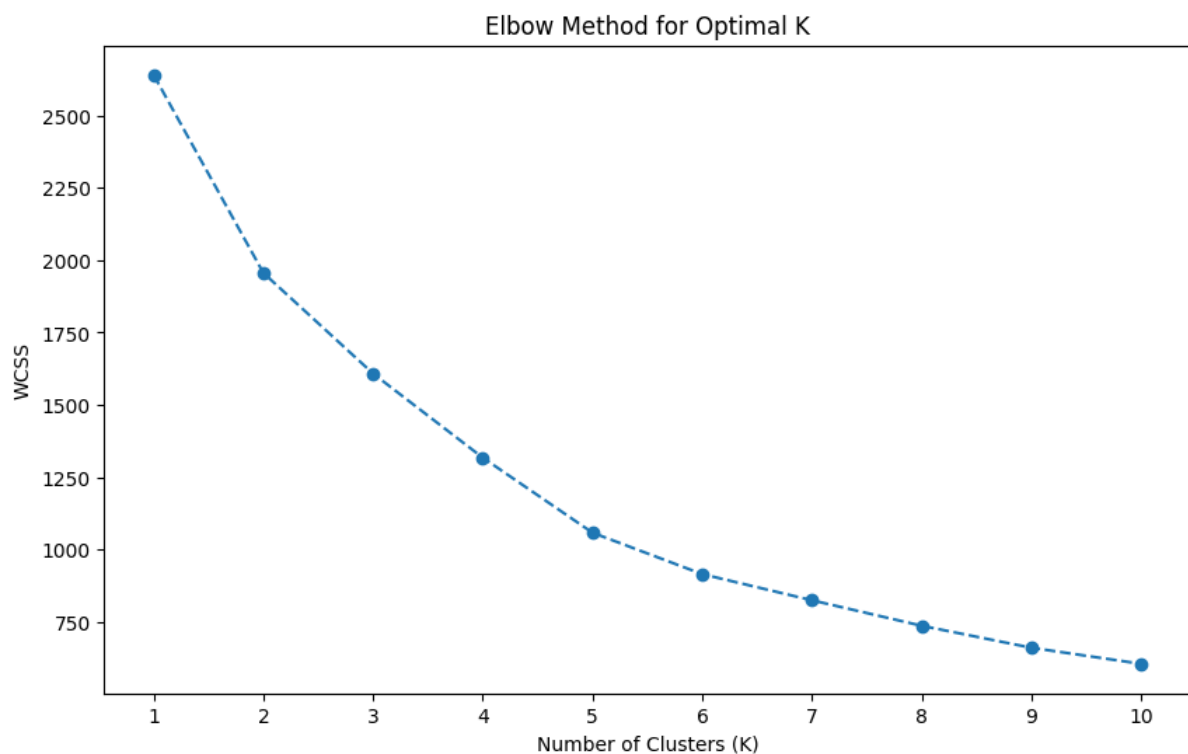
plt.title('Elbow Method for Optimal K')

plt.xlabel('Number of Clusters (K)')

plt.ylabel('WCSS')

plt.xticks(range(1, 11))

plt.show()
```



```
n_clusters_kmeans = 5
```

Step 8.1.2: Train K-Means Model

```
kmeans_model = KMeans(n_clusters=n_clusters_kmeans, init='k-means++', max_iter=300,  
n_init=10,
```

```
random_state=42)
```

```
kmeans_labels = kmeans_model.fit_predict(scaled_data)
```

Step 8.1.3: Evaluate K-Means

```
silhouette_avg_kmeans = silhouette_score(scaled_data, kmeans_labels)
```

```
print(f"K-Means Silhouette Score (k={n_clusters_kmeans}): {silhouette_avg_kmeans:.3f}")
```

Step 8.1.5: Analyze Cluster Centroids (Mean values for each feature in each cluster)

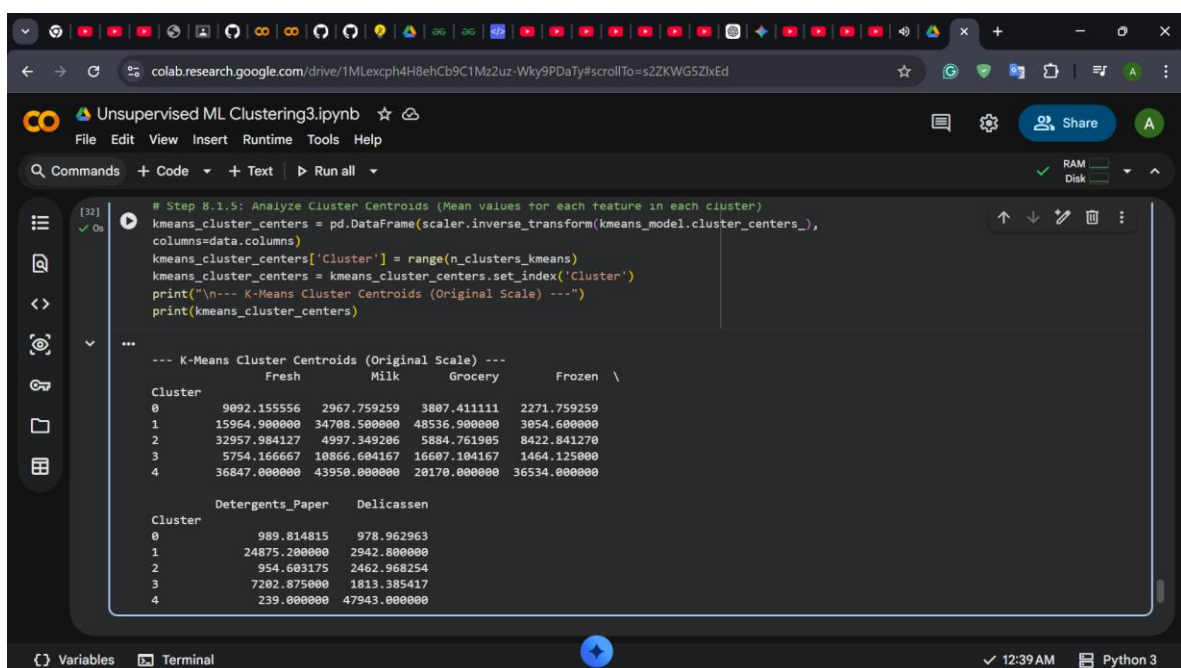
```
kmeans_cluster_centers =  
pd.DataFrame(scaler.inverse_transform(kmeans_model.cluster_centers_),  
columns=data.columns)
```

```
kmeans_cluster_centers['Cluster'] = range(n_clusters_kmeans)
```

```
kmeans_cluster_centers = kmeans_cluster_centers.set_index('Cluster')
```

```
print("\n--- K-Means Cluster Centroids (Original Scale) ---")
```

```
print(kmeans_cluster_centers)
```



```
# Step 8.1.5: Analyze Cluster Centroids (Mean values for each feature in each cluster)
kmeans_cluster_centers = pd.DataFrame(scaler.inverse_transform(kmeans_model.cluster_centers_),
columns=data.columns)
kmeans_cluster_centers['Cluster'] = range(n_clusters_kmeans)
kmeans_cluster_centers = kmeans_cluster_centers.set_index('Cluster')
print("\n--- K-Means Cluster Centroids (Original Scale) ---")
print(kmeans_cluster_centers)
```

	Fresh	Milk	Grocery	Frozen
Cluster				
0	9092.155556	2967.759259	3807.411111	2271.759259
1	15964.900000	34708.500000	48536.900000	3054.600000
2	32957.984127	4997.349206	5884.761905	8422.841270
3	5754.166667	10866.604167	16607.104167	1464.125000
4	36847.000000	43950.000000	20170.000000	36534.000000

	Detergents_Paper	Delicassen
Cluster		
0	989.814815	978.962963
1	24875.200000	2942.800000
2	954.603175	2462.968254
3	7202.875000	1813.385417
4	239.000000	47943.000000

```
# Visualization of K-Means Clusters (using PCA for 2D representation)

# Apply PCA for dimensionality reduction for visualization

pca = PCA(n_components=2)

principal_components = pca.fit_transform(scaled_data)

pca_df = pd.DataFrame(data=principal_components, columns=['PC1', 'PC2'])

pca_df['KMeans_Cluster'] = kmeans_labels

plt.figure(figsize=(10, 8))

sns.scatterplot(x='PC1', y='PC2', hue='KMeans_Cluster', data=pca_df, palette='viridis', s=100,
alpha=0.7)

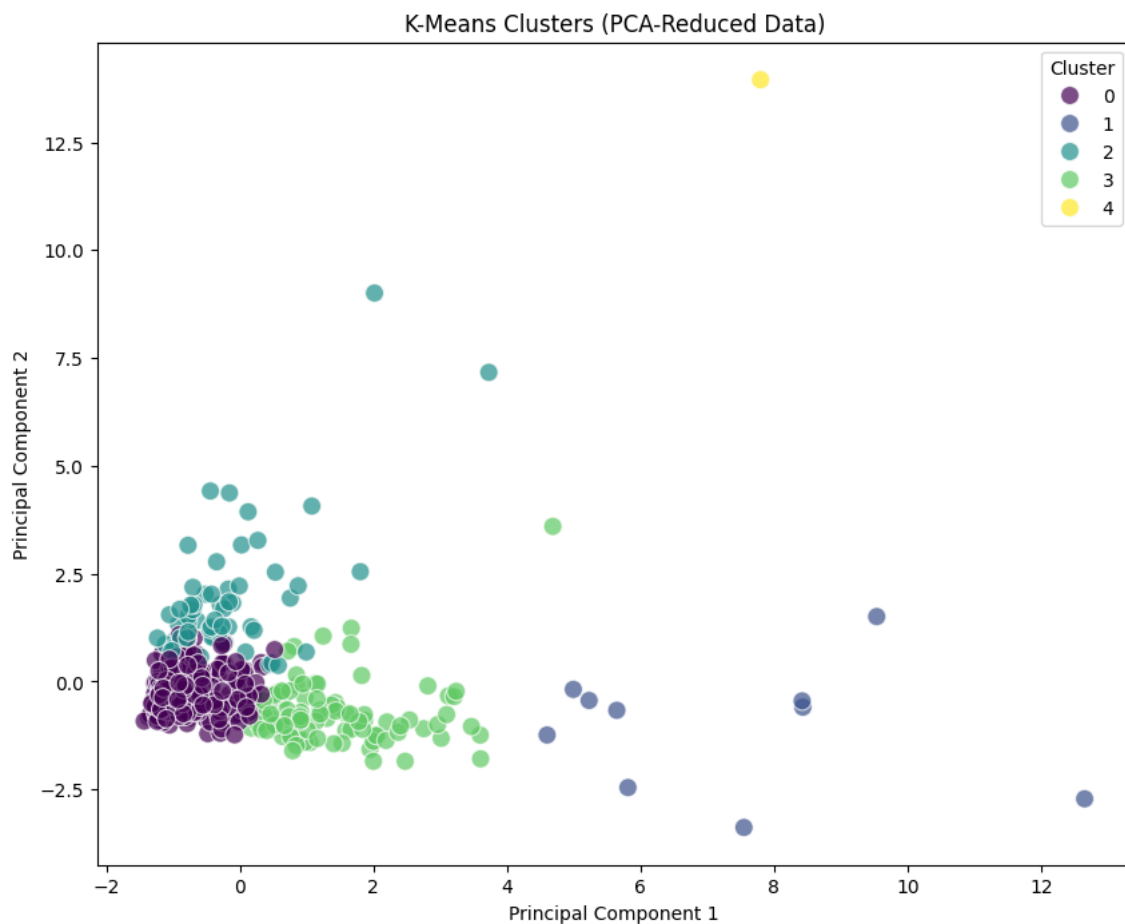
plt.title('K-Means Clusters (PCA-Reduced Data)')

plt.xlabel('Principal Component 1')

plt.ylabel('Principal Component 2')

plt.legend(title='Cluster')

plt.show()
```



8.2 Hierarchical Clustering (Agglomerative)

Step 8.2.1: Create Dendrogram to help choose number of clusters

This can be computationally intensive for very large datasets

```
plt.figure(figsize=(15, 8))
```

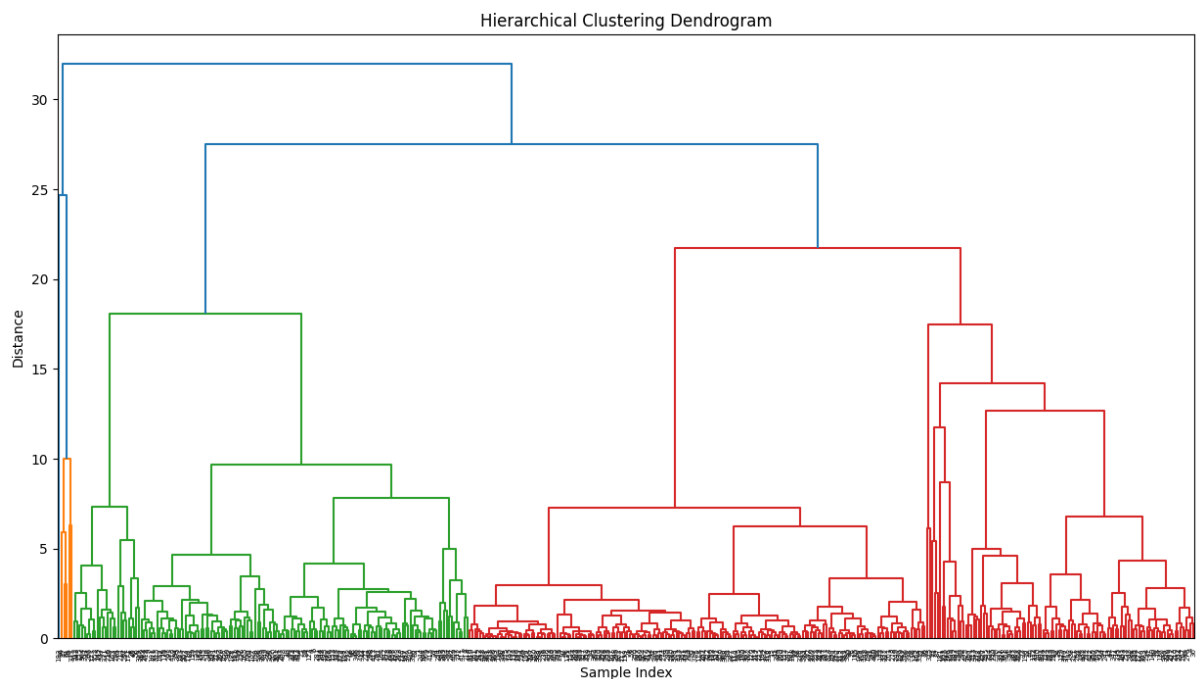
```
dendrogram(linkage(scaled_data, method='ward')) # 'ward' minimizes variance within clusters
```

```
plt.title('Hierarchical Clustering Dendrogram')
```

```
plt.xlabel('Sample Index')
```

```
plt.ylabel('Distance')
```

```
plt.show()
```



Step 8.2.2: Train Agglomerative Clustering Model

```
agg_model = AgglomerativeClustering(n_clusters=n_clusters_hierarchical, linkage='ward')
```

```
agg_labels = agg_model.fit_predict(scaled_data)
```

Step 8.2.3: Evaluate Hierarchical Clustering

```
silhouette_avg_agg = silhouette_score(scaled_data, agg_labels)
```

```
print(f"Hierarchical Clustering Silhouette Score (k={n_clusters_hierarchical}):  
{silhouette_avg_agg:.3f}")
```

colab.research.google.com/drive/1MLexcp4H8ehCb9C1Mz2uz-Wky9PDaTy#scrollTo=_7-VMLxYm8vz

Unsupervised ML Clustering3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Sample Index

```
[37] n_clusters_hierarchical = 5
# Step 8.2.2: Train Agglomerative Clustering Model
agg_model = AgglomerativeClustering(n_clusters=n_clusters_hierarchical, linkage='w
agg_labels = agg_model.fit_predict(scaled_data)

[39] # Step 8.2.3: Evaluate Hierarchical Clustering
silhouette_avg_agg = silhouette_score(scaled_data, agg_labels)
print(f"Hierarchical Clustering Silhouette Score (k={n_clusters_hierarchical}): {s
... Hierarchical Clustering Silhouette Score (k=5): 0.240
```

Step 8.2.5: Analyze Cluster Centroids (Mean values for each feature in each cluster)

For Agglomerative, there are no explicit centroids, so we calculate means for each cluster

```
agg_cluster_centers = df_agg.groupby('Agg_Cluster')[data.columns].mean()
```

```
print("\n--- Hierarchical Cluster Centroids (Original Scale) ---")
```

```
print(agg_cluster_centers)
```

colab.research.google.com/drive/1MLexcp4H8ehCb9C1Mz2uz-Wky9PDaTy#scrollTo=dYlrm3B7nUhb

Unsupervised ML Clustering3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[41] # Step 8.2.5: Analyze Cluster Centroids (Mean values for each feature in each cluster)
# For Agglomerative, there are no explicit centroids, so we calculate means for each cluster
agg_cluster_centers = df_agg.groupby('Agg_Cluster')[data.columns].mean()
print("\n--- Hierarchical Cluster Centroids (Original Scale) ---")
print(agg_cluster_centers)

...
--- Hierarchical Cluster Centroids (Original Scale) ---
Agg_Cluster  Fresh      Milk      Grocery      Frozen \
0          5794.241830  9419.888889  13761.738562  1461.764706
1          25603.000000  43460.000000  61472.200000  2636.000000
2          27256.278846  4914.384615  5931.461538  6277.769231
3          36847.000000  43950.000000  20170.000000  36534.000000
4           7876.254237  1902.627119  2534.536723  2403.378531

Agg_Cluster  Detergents_Paper  Delicassen
0          5913.379085  1577.607843
1          29974.200000  2708.800000
2          1184.971154  2272.942308
3          239.000000  47943.000000
4           507.135593   744.045198
```

Variables Terminal 12:46 AM Python 3

```
# Visualization of Hierarchical Clusters (using PCA for 2D representation)
```

```
pca_df['Agg_Cluster'] = agg_labels
```

```
plt.figure(figsize=(10, 8))
```

```
sns.scatterplot(x='PC1', y='PC2', hue='Agg_Cluster', data=pca_df, palette='plasma', s=100,  
alpha=0.7)
```

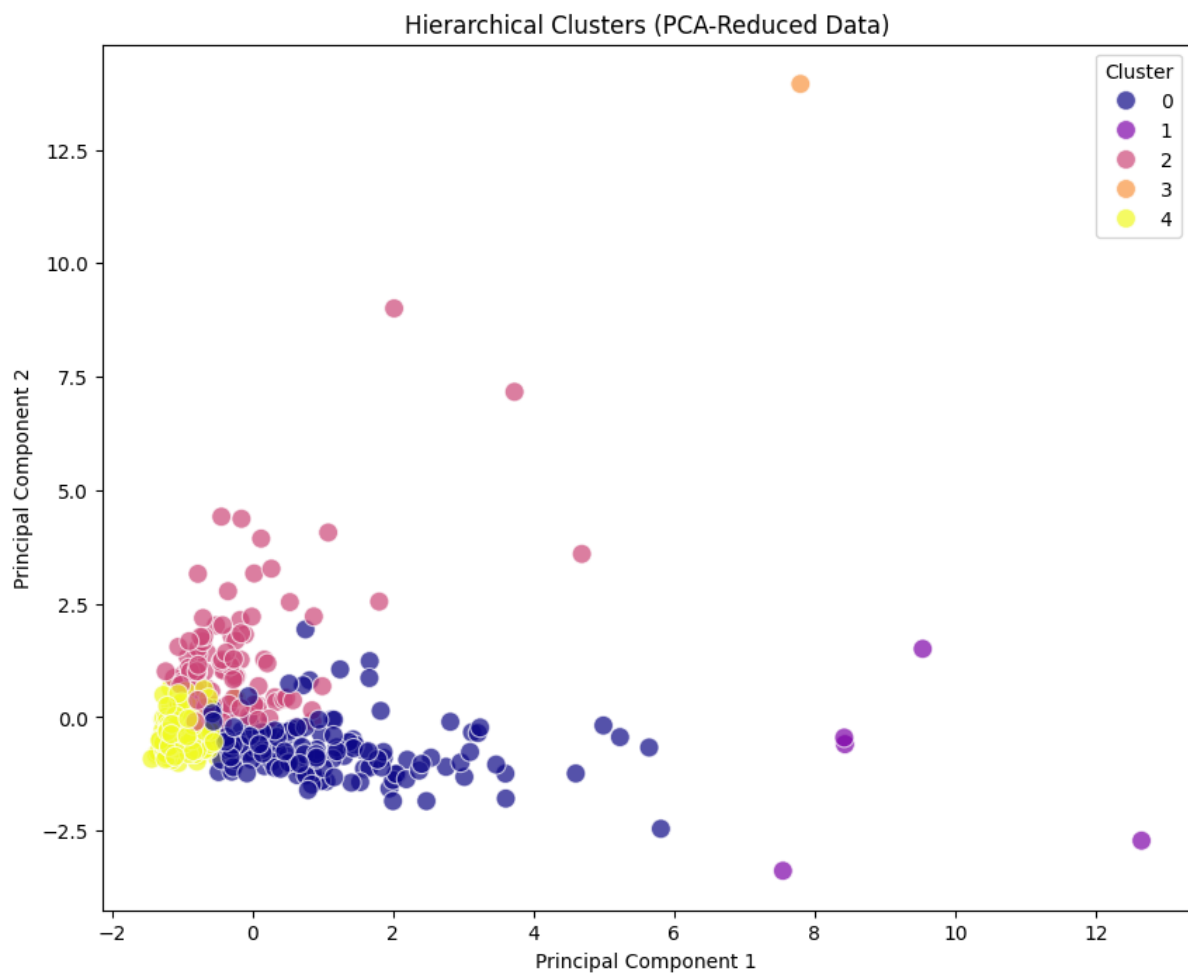
```
plt.title('Hierarchical Clusters (PCA-Reduced Data)')
```

```
plt.xlabel('Principal Component 1')
```

```
plt.ylabel('Principal Component 2')
```

```
plt.legend(title='Cluster')
```

```
plt.show()
```



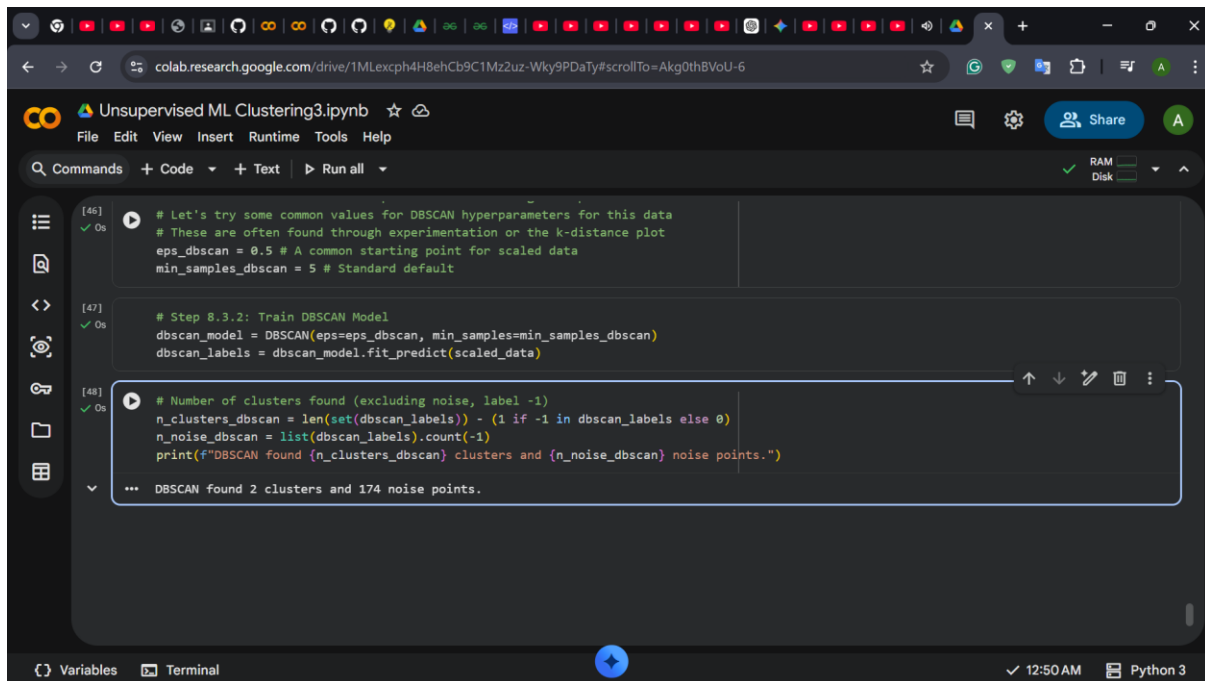
8.3 DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Number of clusters found (excluding noise, label -1)

```
n_clusters_dbscan = len(set(dbscan_labels)) - (1 if -1 in dbscan_labels else 0)
```

```
n_noise_dbscan = list(dbscan_labels).count(-1)
```

```
print(f"DBSCAN found {n_clusters_dbscan} clusters and {n_noise_dbscan} noise points.")
```

A screenshot of a Google Colab notebook titled 'Unsupervised ML Clustering3.ipynb'. The notebook interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a toolbar with icons for commands, code, text, and running, and a sidebar with file and variable views. The main area shows three code cells. The first cell (46) sets hyperparameters: `eps_dbscan = 0.5` and `min_samples_dbscan = 5`. The second cell (47) trains the DBSCAN model: `dbscan_model = DBSCAN(eps=eps_dbscan, min_samples=min_samples_dbscan)` and `dbscan_labels = dbscan_model.fit_predict(scaled_data)`. The third cell (48) calculates the number of clusters and noise points using the provided formulas and prints the result: 'DBSCAN found 2 clusters and 174 noise points.' The bottom status bar shows 'Variables', 'Terminal', '12:50 AM', and 'Python 3'.

Step 8.3.3: Evaluate DBSCAN (Silhouette Score does not work well if many noise points)

Silhouette score excludes noise points (-1 label)

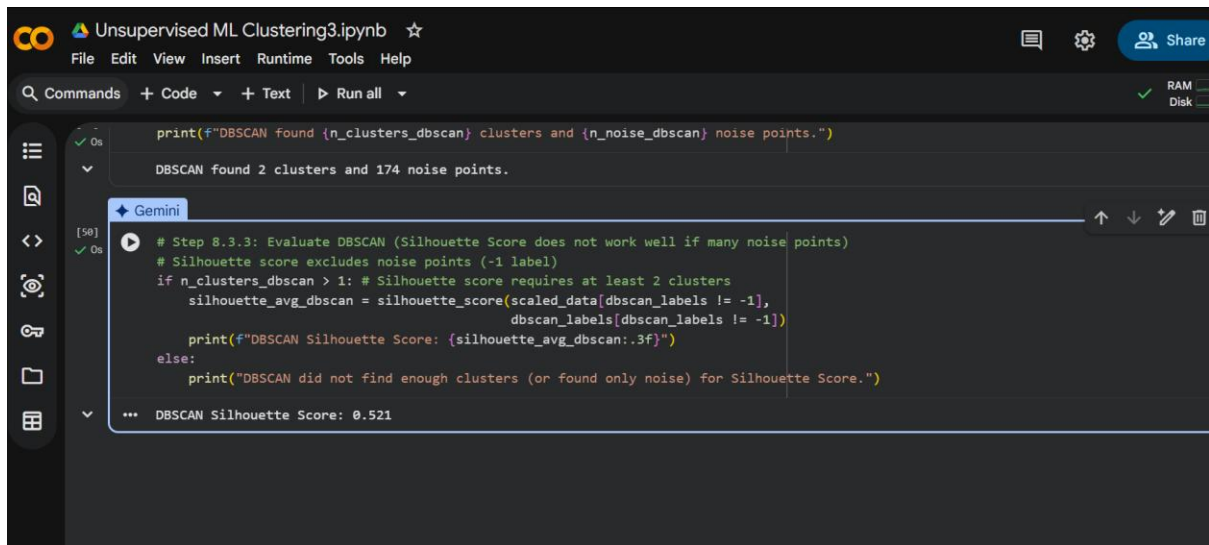
```
if n_clusters_dbscan > 1: # Silhouette score requires at least 2 clusters
```

```
silhouette_avg_dbscan = silhouette_score(scaled_data[dbscan_labels != -1],  
dbscan_labels[dbscan_labels != -1])
```

```
print(f"DBSCAN Silhouette Score: {silhouette_avg_dbscan:.3f}")
```

```
else:
```

```
print("DBSCAN did not find enough clusters (or found only noise) for Silhouette Score.")
```



The screenshot shows a Jupyter Notebook titled "Unsupervised ML Clustering3.ipynb". The code cell [0s] has been executed, displaying the output: "DBSCAN found 2 clusters and 174 noise points." A Gemini code editor overlay is visible, showing the following Python code:

```
# Step 8.3.3: Evaluate DBSCAN (Silhouette Score does not work well if many noise points)
# Silhouette score excludes noise points (-1 label)
if n_clusters_dbSCAN > 1: # Silhouette score requires at least 2 clusters
    silhouette_avg_dbSCAN = silhouette_score(scaled_data[dbSCAN_labels != -1],
                                             dbSCAN_labels[dbSCAN_labels != -1])
    print(f"DBSCAN Silhouette Score: {silhouette_avg_dbSCAN:.3f}")
else:
    print("DBSCAN did not find enough clusters (or found only noise) for Silhouette Score.")
... DBSCAN Silhouette Score: 0.521
```

Analyze Cluster Centroids (Mean values for each feature in each cluster) - excluding noise

if `n_clusters_dbSCAN > 0`:

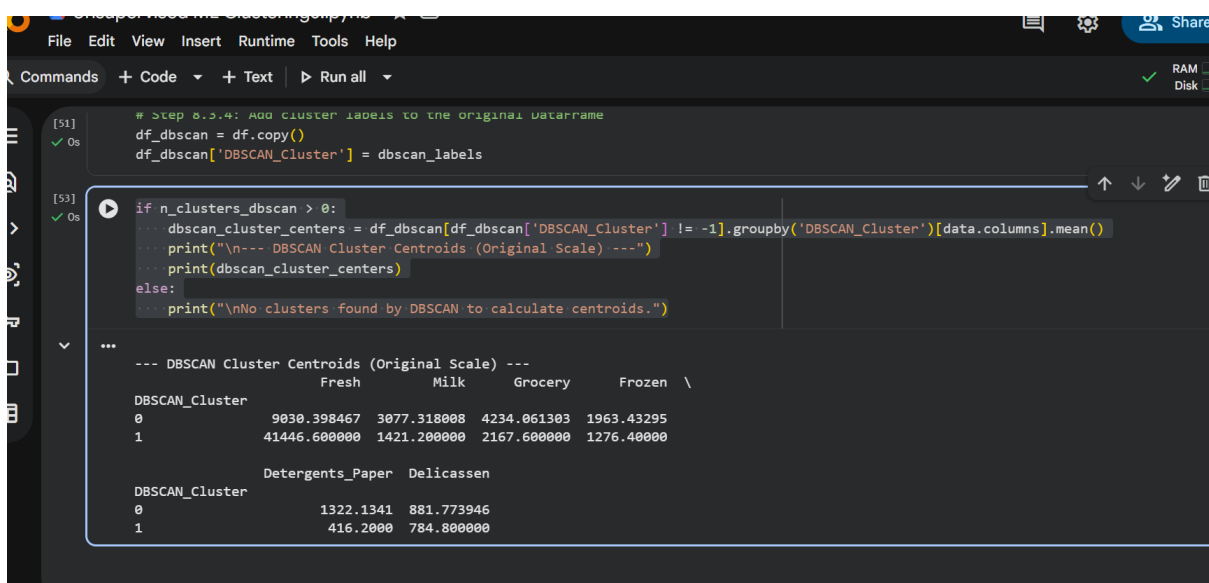
```
dbSCAN_cluster_centers = df_dbSCAN[df_dbSCAN['DBSCAN_Cluster'] != -1].groupby('DBSCAN_Cluster')[data.columns].mean()
```

```
print("\n--- DBSCAN Cluster Centroids (Original Scale) ---")
```

```
print(dbSCAN_cluster_centers)
```

else:

```
print("\nNo clusters found by DBSCAN to calculate centroids.")
```



The screenshot shows a Jupyter Notebook titled "Unsupervised ML Clustering3.ipynb". The code cell [51] has been executed, displaying the output: "DBSCAN found 2 clusters and 174 noise points." A Gemini code editor overlay is visible, showing the following Python code:

```
# Step 8.3.4: Add cluster labels to the original dataframe
df_dbSCAN = df.copy()
df_dbSCAN['DBSCAN_Cluster'] = dbSCAN_labels

if n_clusters_dbSCAN > 0:
    dbSCAN_cluster_centers = df_dbSCAN[df_dbSCAN['DBSCAN_Cluster'] != -1].groupby('DBSCAN_Cluster')[data.columns].mean()
    print("\n--- DBSCAN Cluster Centroids (Original Scale) ---")
    print(dbSCAN_cluster_centers)
else:
    print("\nNo clusters found by DBSCAN to calculate centroids.")
...
--- DBSCAN Cluster Centroids (Original Scale) ---

```

	Fresh	Milk	Grocery	Frozen
DBSCAN_Cluster				
0	9030.398467	3077.318008	4234.061303	1963.43295
1	41446.600000	1421.200000	2167.600000	1276.400000

	Detergents_Paper	Delicassen
DBSCAN_Cluster		
0	1322.1341	881.773946
1	416.2000	784.800000

```
# Visualization of DBSCAN Clusters (using PCA for 2D representation)
```

```
pca_df['DBSCAN_Cluster'] = dbscan_labels
```

```
plt.figure(figsize=(10, 8))
```

```
# Plot noise points in a distinct color/marker
```

```
sns.scatterplot(x='PC1', y='PC2', hue='DBSCAN_Cluster',  
data=pca_df[pca_df['DBSCAN_Cluster'] != -1],
```

```
palette='cubehelix', s=100, alpha=0.7, legend='full')
```

```
sns.scatterplot(x='PC1', y='PC2', data=pca_df[pca_df['DBSCAN_Cluster'] == -1],
```

```
color='gray', marker='X', s=50, label='Noise (-1)', alpha=0.5)
```

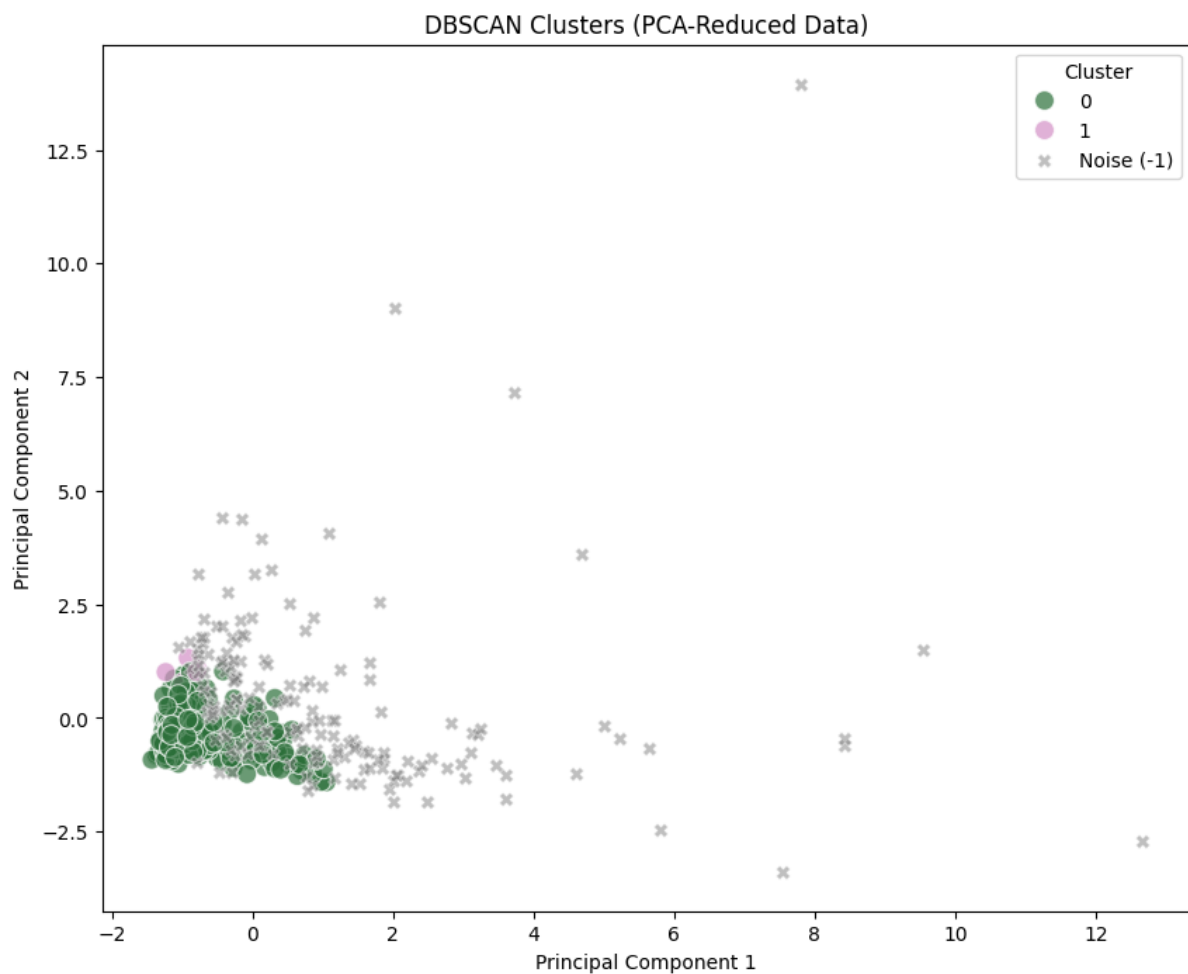
```
plt.title('DBSCAN Clusters (PCA-Reduced Data)')
```

```
plt.xlabel('Principal Component 1')
```

```
plt.ylabel('Principal Component 2')
```

```
plt.legend(title='Cluster')
```

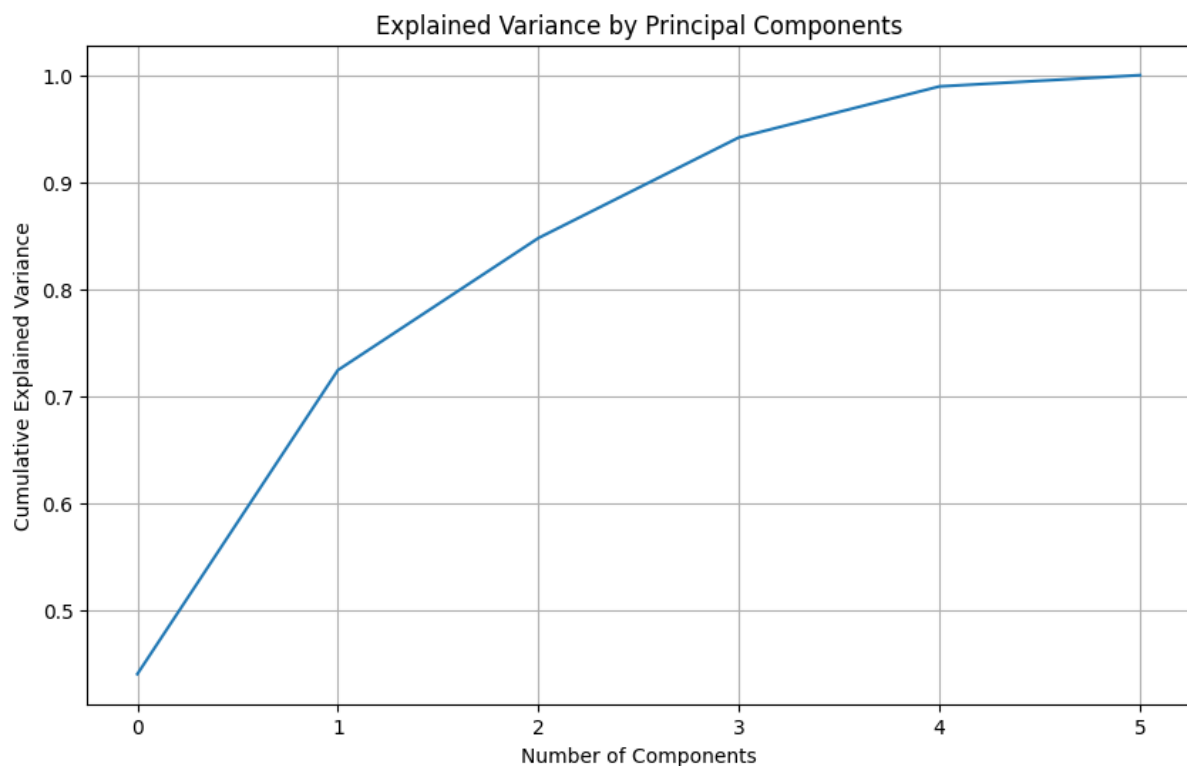
```
plt.show()
```



8.4 Principal Component Analysis (PCA) - Dimensionality Reduction

Step 8.4.2: Plot Explained Variance Ratio

```
plt.figure(figsize=(10, 6))  
plt.plot(np.cumsum(pca_full.explained_variance_ratio_))  
plt.xlabel('Number of Components')  
plt.ylabel('Cumulative Explained Variance')  
plt.title('Explained Variance by Principal Components')  
plt.grid(True)  
plt.show()
```



Based on the plot, 2-3 components explain a good portion of the variance (e.g., 2 components explain ~60%)

```
n_components_pca = 2 # For visualization and simplicity  
pca = PCA(n_components=n_components_pca)  
principal_components = pca.fit_transform(scaled_data)  
print(f"\nExplained Variance Ratio for {n_components_pca} components:")  
print(pca.explained_variance_ratio_)
```

[illegible]

```
# Visualization of PCA-reduced data itself (coloring by K-Means labels from PCA)
```

```
plt.figure(figsize=(10, 8))
```

```
sns.scatterplot(x='PC1', y='PC2', hue='KMeans_Cluster_on_PCA', data=pca_df_final,
palette='viridis', s=100, alpha=0.7)

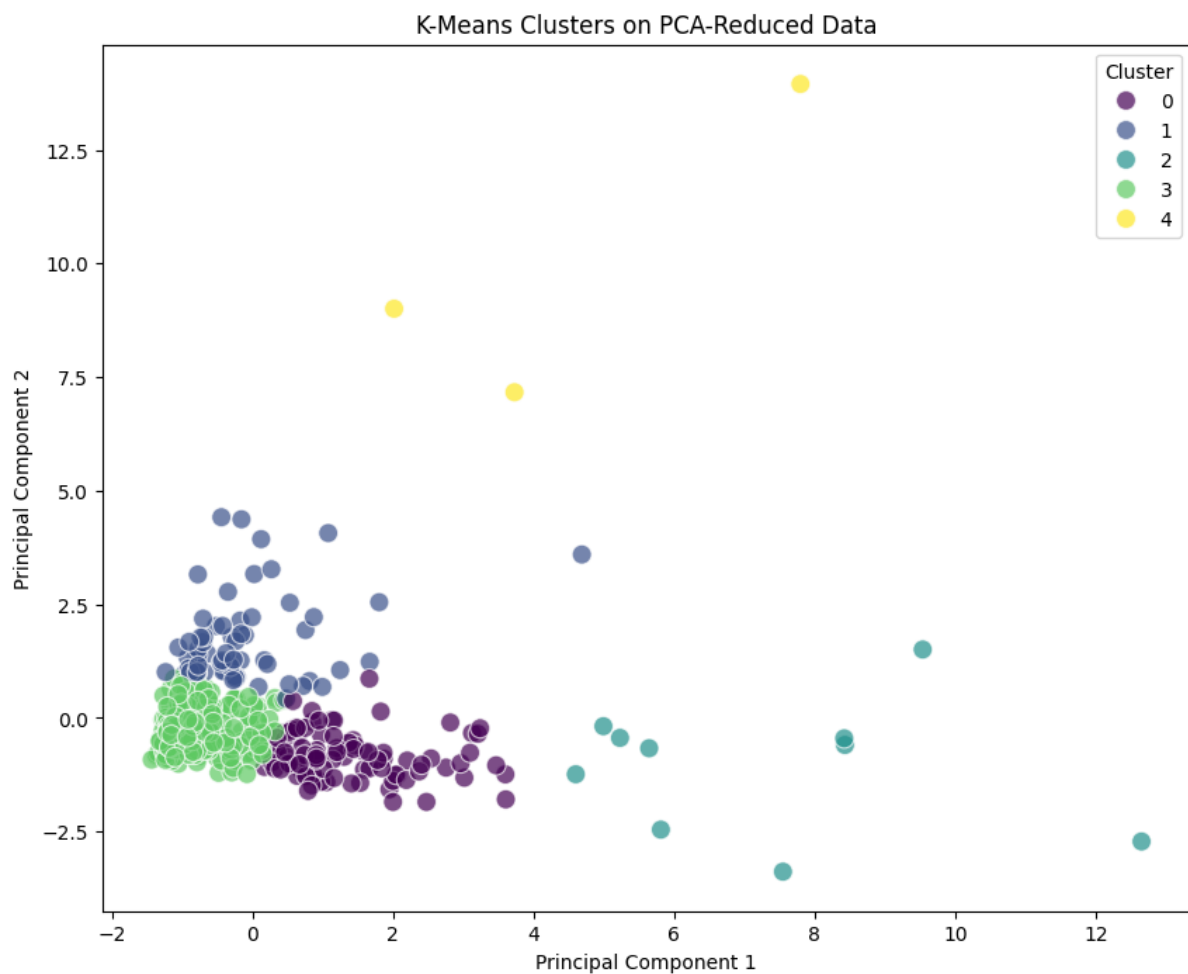
plt.title('K-Means Clusters on PCA-Reduced Data')

plt.xlabel('Principal Component 1')

plt.ylabel('Principal Component 2')

plt.legend(title='Cluster')

plt.show()
```

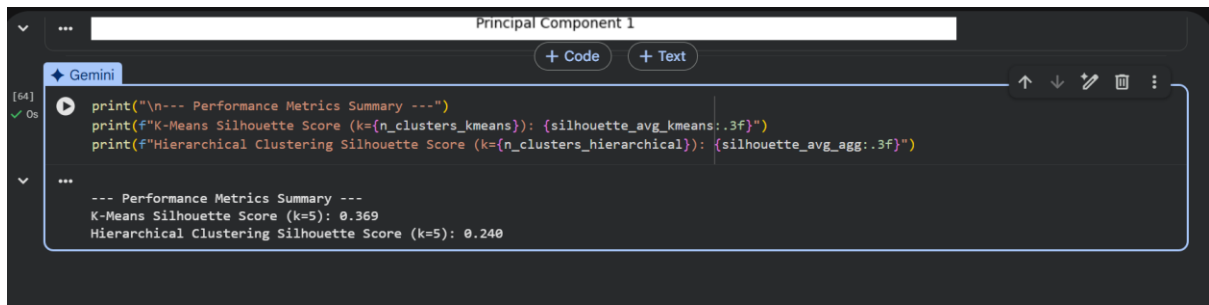


9. Performance Metrics & Evaluation

```
print("\n--- Performance Metrics Summary ---")

print(f"K-Means Silhouette Score (k={n_clusters_kmeans}): {silhouette_avg_kmeans:.3f}")

print(f"Hierarchical Clustering Silhouette Score (k={n_clusters_hierarchical}): {silhouette_avg_agg:.3f}")
```



The screenshot shows a Jupyter Notebook interface. At the top, there's a tab labeled 'Principal Component 1'. Below it, a code cell is selected, showing Python code for printing performance metrics. The code includes comments and f-strings for K-Means and Hierarchical Clustering silhouette scores. The output of the cell is displayed below the code, showing the same metrics with numerical values.

```
[64]: print("\n--- Performance Metrics Summary ---")
      print(f"K-Means Silhouette Score (k={n_clusters_kmeans}): {silhouette_avg_kmeans:.3f}")
      print(f"Hierarchical Clustering Silhouette Score (k={n_clusters_hierarchical}): {silhouette_avg_agg:.3f}")

      --- Performance Metrics Summary ---
      K-Means Silhouette Score (k=5): 0.369
      Hierarchical Clustering Silhouette Score (k=5): 0.240
```

```
if n_clusters_dbscan > 1:
```

```
    print(f"DBSCAN Silhouette Score: {silhouette_avg_dbscan:.3f} (Note: Excludes noise points)")
```

```
else:
```

```
    print("DBSCAN did not find enough clusters for meaningful Silhouette Score.")
```

DBSCAN Silhouette Score: 0.521 (Note: Excludes noise points)

10. Prediction and Decision Making

```
# Create a hypothetical new customer's spending data
```

```
new_customer_data = pd.DataFrame([[10000, 5000, 8000, 1000, 2000, 500]],
```

```
columns=data.columns)
```

```
print("\n--- New Customer Data ---")
```

```
print(new_customer_data)
```

```
DBSCAN Silhouette Score: 0.321 (NOTE: EXCLUDES NOISE POINTS)

[67] ✓ Os
on PCA data for comparison
-Means Silhouette Score on PCA-reduced data: {silhouette_avg_kmeans_pca:.3f}")

K-Means Silhouette Score on PCA-reduced data: 0.488

[68] ✓ Os
# Create a hypothetical new customer's spending data
new_customer_data = pd.DataFrame([[10000, 5000, 8000, 1000, 2000, 500]],
columns=data.columns)
print("\n--- New Customer Data ---")
print(new_customer_data)

...

--- New Customer Data ---
   Fresh Milk Grocery Frozen Detergents_Paper Delicassen
0  10000  5000    8000   1000             2000           500
```

Step 1: Scale the new customer data using the SAME scaler fitted before

```
scaled_new_customer = scaler.transform(new_customer_data)
```

```
print("\n--- Scaled New Customer Data ---")
```

```
print(scaled_new_customer)
```

Step 2: Predict the cluster for the new customer using K-Means

```
new_customer_kmeans_cluster = kmeans_model.predict(scaled_new_customer)
```

```
print(f"\nNew customer assigned to K-Means Cluster: {new_customer_kmeans_cluster[0]}")
```

```
--- New Customer Data ---
   Fresh Milk Grocery Frozen Detergents_Paper Delicassen
0  10000  5000    8000   1000             2000           500

[69] ✓ Os
# Step 1: Scale the new customer data using the SAME scaler fitted before
scaled_new_customer = scaler.transform(new_customer_data)
print("\n--- Scaled New Customer Data ---")
print(scaled_new_customer)

...

--- Scaled New Customer Data ---
[[[-0.15833973 -0.10881241  0.00513284 -0.42727701 -0.18509302 -0.36382926]]

[70] ✓ Os
# Step 2: Predict the cluster for the new customer using K-Means
new_customer_kmeans_cluster = kmeans_model.predict(scaled_new_customer)
print(f"\nNew customer assigned to K-Means Cluster: {new_customer_kmeans_cluster[0]}")

...

New customer assigned to K-Means Cluster: 0
```

1. What you have learned at a glance I learned how to implement unsupervised machine learning techniques, specifically clustering, to identify hidden patterns in unlabeled datasets. Using the 'Wholesale Customers' data, and mastered the process of grouping customers based on their annual spending habits across various product categories.

2. What difficulties you faced during the learning process The main challenges involved handling initial coding errors, such as 'IndentationError' and issues with file paths when loading the dataset from local directories. Another one, determining the optimal number of clusters and understanding the mathematical intuition behind dimensionality reduction (PCA) required careful troubleshooting and research.

3. What skill(s) you gained you think I gained technical proficiency in data pre-processing (scaling and normalization) and dimensionality reduction using PCA. I also developed practical skills in applying various clustering algorithms including K-Means, Hierarchical Clustering, and DBSCAN and evaluating their performance using Silhouette Scores.

4. How you could apply these skills in your workplace If hired, I can use these clustering techniques to perform market segmentation, allowing the company to identify high-value customer groups. This data-driven approach would help in tailoring marketing strategies and optimizing resource allocation to improve overall business efficiency and customer satisfaction.

5. Write your motivation towards further learning and exploration I am motivated by the ability of data science to transform raw, complex data into actionable business insights. I am eager to explore more advanced neural networks and deep learning models to solve larger scale real world problems and contribute to evidence based decision making in a professional environment.